

torch.cuda.jiterator._create_jit_fn#

https://pytorch.org/docs/stable/generated/torch.cuda.jiterator._create_jit_fn.html

Description:

Create a jiterator-generated cuda kernel for an elementwise op. The code string has to be a valid CUDA function that describes the computation for a single element. The code string has to follow the c++ template pattern, as shown in the example below. This function will be inlined into elementwise kernel template, and compiled on the fly. Compiled kernel will be cached in memory, as well as local temp dir. Jiterator-generated kernels accepts noncontiguous tensors, and supports broadcasting and type promotion. `code_string (str)` – CUDA code string to be compiled by jiterator. The entry functor must return by value. `kwargs (Dict, optional)` – Keyword arguments for generated function

Function Signature

```
torch.cuda.jiterator._create_jit_fn(code_string, **kwargs)  
[source]#
```

Parameters

Return type

Returns:

Callable

Code Examples

```
code_string = "template <typename T> T my_kernel(T x, T y, T  
alpha) { return -x + alpha * y; }"  
jitted_fn = create_jit_fn(code_string, alpha=1.0)  
a = torch.rand(3, device="cuda")  
b = torch.rand(3, device="cuda")  
# invoke jitted function like a regular python function  
result = jitted_fn(a, b, alpha=3.14)
```

```

code_string = (
    "template <typename T> T util_fn(T x, T y) { return ::sin(x)
+ ::cos(y); }"
)
code_string += "template <typename T> T my_kernel(T x, T y, T
val) { return ::min(val, util_fn(x, y)); }"
jitted_fn = create_jit_fn(code_string, val=0.0)
a = torch.rand(3, device="cuda")
b = torch.rand(3, device="cuda")
# invoke jitted function like a regular python function
result = jitted_fn(a, b) # using default val=0.0

```

```

code_string = "template <typename T> T my_gelu(T a) { return a >
0 ? a : 0; }"
my_gelu = create_jit_fn(code_string)
my_lib = torch.library.Library("aten", "IMPL")
my_lib.impl("aten::gelu", my_gelu, "CUDA")
# torch.nn.GELU and torch.nn.functional.gelu are now overridden
a = torch.rand(3, device="cuda")
torch.allclose(torch.nn.functional.gelu(a),
torch.nn.functional.relu(a))

```

Notes & Warnings

 **WARNING** Warning This API is in beta and may change in future releases.

 **WARNING** Warning This API only supports up to 8 inputs and 1 output

 **WARNING** Warning All input tensors must live in CUDA device

Detailed Documentation

Documentation

Create a jiterator-generated cuda kernel for an elementwise op.

The code string has to be a valid CUDA function that describes the computation for a single element. The code string has to follow the c++ template pattern, as shown in the example below. This function will be inlined into elementwise kernel template, and compiled on the fly. Compiled kernel will be cached in memory, as well as local temp dir.

Jiterator-generated kernels accepts noncontiguous tensors, and supports broadcasting and type promotion.

code_string (str) – CUDA code string to be compiled by jiterator. The entry functor must

return by value.

`kwargs` (Dict, optional) – Keyword arguments for generated function

`code_string` also allows multiple function definitions, and the last function will be treated as the entry function.

Jiterator can be used together with python registration to override an operator's cuda kernel. Following example is overriding gelu's cuda kernel with relu.

This API is in beta and may change in future releases.

This API only supports up to 8 inputs and 1 output

All input tensors must live in CUDA device